Team 13 maintenance plan

Prepared by Bhavesh Velluru, 9 November 2026. Final source hashes are in docs/evidence/final/source-manifest.json. Proposed operating responsibilities follow the approved module ownership; response targets below are team proposals for review.

Maintenance responsibilities

Type	Trigger and work	Owner	Verification
Corrective	Reproduce a reported fault, link requirement/test/defect, fix and retest	Affected module owner	Failing case passes and affected regression suite stays green
Adaptive	Python/framework, browser, database, Redis, Docker or model-format changes	M3 for runtime, M2 for ingestion, M4 for reports/UI	Rebuild pinned images; compatibility fixtures and Compose smoke
Perfective	Improve workflow, chart clarity or comparison using observed feedback	M2 + M4	Acceptance criteria, browser tests and participant feedback
Preventive	Review dependencies, secrets, orphan jobs, retention and quality findings	All; M3 coordinates runtime	CI scans, containment suite, cleanup and retention tests

M1 owns accounts, role policy and workspaces; M2 owns files and compatibility; M3 owns queue, inference and recovery; M4 owns charts, comparison, export and this documentation. Each change needs a reviewer from another module and an updated test/evidence reference.

Defect and change process

Record severity, reproduction steps, expected/actual result, environment, affected requirement, owner and evidence in an issue. Use Open -> In progress -> Fixed -> Retest -> Closed. A PR links the issue, explains schema/API changes and includes the necessary regression check. Review dependency impact before changing an API shared with another module.

Proposed targets are immediate triage for Critical containment, disclosure or data-loss incidents; same-working-day triage for High failures; and backlog prioritization at the next review for Medium/Low findings. Resolution timing depends on reproduction and review. A security or containment failure suspends further untrusted execution until the policy and regression tests pass.

Deployment, recovery and rollback

1. Record the known-good commit and image tags, configuration names and migration state. Keep secret values outside Git and logs.

2. Before a schema change, back up PostgreSQL with pg_dump and copy the media volume with access restricted to the maintainers. Preserve checksum/provenance records with the backup.

3. Build the candidate images, apply migrations, run API/browser smoke checks and a real sandbox evaluation, and check worker/beat and queue health.

4. If checks fail, stop new evaluation submissions, preserve diagnostics and deploy the known-good images. Assess migration reversibility before reverting schema; restore the verified database/media backup when a backwards-compatible rollback is unavailable.

5. Verify login, history, upload/compatibility, one evaluation and report download after recovery. Record recovery duration and any lost jobs.

Backup frequency is proposed as daily for an actively used deployment and before every migration. Retention and storage location must be approved by the team before production use. Rehearse a restore on a disposable environment; a backup file alone is insufficient recovery evidence.

Routine checks

Frequency / event	Action
Every PR	Ruff, migrations, tests/coverage, dependency/static/secret scans, review
Every deployment	Compose startup, auth, upload, real sandbox evaluation, report download and orphan check
Daily during use	Queue age, failed/timed-out jobs, audit events, disk usage and backup outcome
Weekly during maintenance	Review vulnerabilities, stale issues, risk owners and dependency updates
After runtime changes	Sandbox policy, hostile-model suite, worker recovery and supported model fixtures
After UI changes	Main workflow, responsive layout, unauthorized paths and chart/report checks

Retention and privacy

RETENTION_DAYS defaults to 30. The merged beat schedule invokes ingestion.purge_expired hourly. Active evaluations retain their inputs until they become terminal. Evaluation history keeps checksum/metric snapshots after upload expiry. Deleting the last referencing evaluation now removes its uploads, while inputs shared with other runs are preserved. Physical file removal follows database commit. Restrict backup access and apply the agreed deletion policy to backups as well as live files.

Initial maintenance backlog

Final code resolves evaluator access, React controls, upload deletion and HTTPS configuration. Keep the documented ADMIN hierarchy under review. Consult FINAL_SUBMISSION_CHECKLIST.md for the actual final quality results and outstanding participant, Windows/browser and video evidence. Retest these boundaries after future changes.